PROJECT TESTING PHASE

Project Title: ComicCraft – AI Comic Story Creator using Gemini Models

College Name: Government Arts and Science College, Manur.

College Code: MSU142

Department: B.SC Computer Science

Team ID: 6ab214cd186ba30941b6117f

Team leader : Swetha D NMID:DC7909570E81C629D5840AFD7E3314CD

Team member : Santhi S NM ID:3A2360143DF92A195DF45E8B245C0012

Team member : Jayaabitha A.S NMID:AAEE35007C90E8898C44BEBE43775768

1. Introduction

The testing phase of ComicCraft checks whether the AI comic story creator works according to its requirements. Testing focuses on user input, story generation, panel formatting, editing, error handling, usability, and basic security. The purpose is to identify problems and correct them before the project is demonstrated or delivered.

2. Objectives of Testing

- Verify that the application accepts valid story prompts.
- Check that Gemini generates relevant and coherent comic stories.
- Confirm that the generated story is divided into the requested number of panels.
- Verify that each panel includes scene descriptions and suitable dialogue or captions.
- Check editing, regeneration, and copy/export features.
- Identify errors and improve reliability, usability, and security.

3. Scope of Testing

Testing covers the user interface, input validation, prompt construction, Gemini integration, response formatting, panel count, editing features, error messages, and

basic privacy practices. Image generation is tested only if it is included in the implemented version; otherwise, illustration prompts are checked as text.

4. Types of Testing

Testing Type	Purpose
Functional Testing	Checks whether each feature works as expected.
Input Validation Testing	Checks empty prompts, long prompts, and missing selections.
Integration Testing	Checks communication between the application and Gemini service.
Usability Testing	Checks whether users can understand and use the interface easily.
Error Handling Testing	Checks network failures, API errors, and invalid responses.
Security Testing	Checks that API credentials are protected and unnecessary personal data is not collected.
Compatibility Testing	Checks the application in supported browsers and screen sizes.

5. Test Environment

- Hardware: Computer or laptop; mobile device may be used for responsive testing.
- Software: Supported web browser and the project development environment.
- Application: ComicCraft prototype.
- AI service: Configured Gemini model/API, or a mock response when API access is unavailable.
- Network: Internet connection for live Gemini integration testing.

6. Test Cases

The following test cases are planned for the prototype. The status column should be updated after each test is actually performed.

Test ID	Test Input / Action	Expected Result	Status

TC01	Enter a valid comic topic	A story is generated based on the topic.	Not tested
TC02	Submit an empty topic	A clear validation message appears.	Not tested
TC03	Choose 5 panels	The result contains 5 panels.	Not tested
TC04	Choose a different panel count	The result follows the selected count.	Not tested
TC05	Select a genre	The story reflects the selected genre.	Not tested
TC06	Check panel details	Each panel has a scene and dialogue/caption where appropriate.	Not tested
TC07	Use the edit feature	The user can modify generated text.	Not tested
TC08	Use regenerate feature	A new version is generated when requested.	Not tested
TC09	Copy or export the result	The comic content can be copied or saved.	Not tested
TC10	Simulate network/API failure	A helpful error message and retry option appear.	Not tested
TC11	Test a long prompt	The app handles it safely or asks the user to shorten it.	Not tested
TC12	Check API key handling	The secret is not exposed in public frontend code.	Not tested

7. Testing Procedure

1. Start the ComicCraft application in the test environment.

2. Enter a topic and select the genre and number of panels.
3. Generate the comic and inspect the story, characters, dialogue, and panel order.
4. Repeat with different topics, genres, and panel counts.
5. Test invalid inputs and simulate API or network errors where possible.
6. Try editing, regenerating, copying, and exporting the result.
7. Record actual results, mark each test as Pass or Fail, and note any defects.
8. Correct defects and repeat the relevant tests.

8. Defect Reporting

Defect ID	Description	Severity	Resolution / Status
BUG01	Story does not follow selected panel count	Medium	To be recorded
BUG02	Generated dialogue is missing or misplaced	Medium	To be recorded
BUG03	API/network error is not clearly shown	High	To be recorded
BUG04	Copy/export feature does not work	Medium	To be recorded

Note: These are example defects to look for, not confirmed defects in the application.

9. Acceptance Criteria

- The application accepts valid prompts and provides a readable story.
- The generated output follows the requested panel count and selected genre as closely as possible.
- Panels are presented in a logical order with relevant descriptions and dialogue.
- Invalid input and service failures are handled with understandable messages.
- Editing and copy/export features work if included in the prototype.
- No API secret is exposed in client-side code.

10. Test Results Summary

Metric	Result

Total test cases	12 planned
Passed	To be filled after testing
Failed	To be filled after testing
Blocked / Not run	To be filled after testing
Open defects	To be filled after testing

11. Limitations

- AI-generated stories may vary between runs even with the same prompt.
- Quality depends on prompt clarity, model behavior, and service availability.
- Live integration tests require valid API access and an internet connection.
- Image output cannot be evaluated if the project only generates illustration prompts.

12. Conclusion

The testing phase is an important step in ensuring that ComicCraft is usable and dependable. The planned test cases cover the main features, input validation, AI integration, output quality, error handling, and basic security. Actual results should be recorded during testing, defects should be corrected, and the relevant tests should be repeated before the final project demonstration.